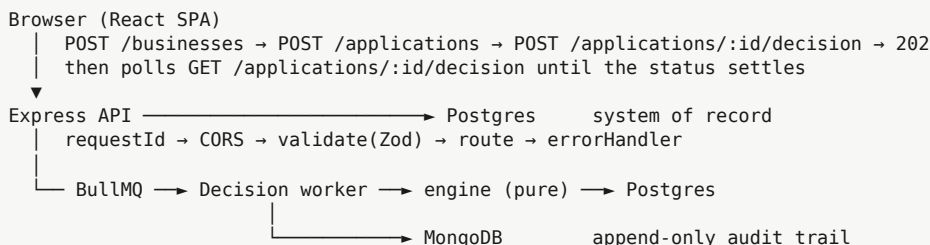


MSME Lending Decision System

Architecture, trade-offs, and what I would change with more time

A lending system's hard problem is not computing a score. It is being able to *defend* one — to an applicant who was declined, to a credit officer retuning the policy, and to a regulator asking why this file was approved eighteen months ago. Nearly every decision below follows from that.

1. Architecture



The engine is a pure function

`evaluate(input)` performs no I/O — no database, no clock, no environment. Three things follow. The entire credit policy is testable without infrastructure, which is why 73 of the 131 tests run in 58ms and probe *both sides* of every threshold boundary. Any historical decision can be reproduced exactly from its stored inputs. And the same function serves the HTTP layer, the queue worker, and the binary search that finds the largest approvable amount, unadapted.

Hard rules are separated from the scorecard

Hard rules — a request above 36× revenue, an instalment exceeding all income, revenue below the lending floor — are checked first and short-circuit scoring entirely. The separation is the point: a scorecard expresses *how much worse does this make the file*, a hard rule expresses *this file is not lendable, full stop*. Collapsing them into one weighted sum is how scorecards approve applications nobody intended to, because enough small positives outvoted one disqualifying fact. A test asserts an otherwise perfect application still fails on a hard rule.

Every threshold lives in one file

`config/scoring.ts` holds every number the engine uses, each with its reasoning. Retuning never means reading business logic. Decisions carry an `engineVersion`, so when those numbers change, past decisions stay interpretable.

2. Trade-offs

Validation errors versus credit rejections

Malformed input is a client error. An unaffordable loan is a credit decision.

The brief's example — ₹10L monthly revenue against a ₹5Cr request — is schema-valid. So it returns `200` with a `REJECTED` decision and `DATA_INCONSISTENCY`, not a `422`.

Returning `422` fails twice over: it tells the client it sent a bad request when it sent a well-formed one, and the rejection never becomes a decision — no score, no reason codes, no audit record, nothing to explain to the applicant or a regulator. An integration test asserts the status code so this cannot regress.

Asynchronous decisions for a synchronous computation

The engine is pure arithmetic and could return inline. It is a queued BullMQ job because of what it becomes: a production engine calls a bureau, pulls GST filings and parses bank statements, any of which takes seconds and can fail and need retrying. Building the boundary now means those slot in without breaking a contract clients depend on — and retrofitting an async boundary onto a synchronous API means breaking every consumer. The worker is its own process, so a slow evaluation cannot consume the API's event loop.

Two databases

Postgres is the system of record: `business → application → decision → reasons` is a real relational chain with referential integrity and money needing exact arithmetic. Reason codes are a normalised child table rather than JSONB so they stay queryable — "*how many declines cited HIGH_EMI_BURDEN this month?*" is a question credit teams ask. MongoDB holds an append-only audit stream whose documents vary by event type and are read only by application id or time range. Being straight about it: **at this scale one Postgres with a JSONB column would do**, with one fewer service to operate. The split suits the shape of the data and the brief specified both, but it is not yet load-bearing.

Audit writes cannot fail a request

Audit writes are fire-and-forget and swallow their errors — a borrower should not be denied a decision because a log store is unreachable. That makes the trail best-effort rather than guaranteed. A system under a retention obligation would invert it: write the audit record inside the decision transaction and fail the request if it could not persist.

Drizzle instead of Prisma

Prisma ships no query-engine binary for `linux-nixos`, this project's development platform, so it cannot generate a client there at all. Its `queryCompiler` preview removes the query engine but migrations still need the schema engine, and pointing at a `/nix/store` path would hard-code an absolute path that breaks for anyone else cloning the repo. Drizzle is pure TypeScript, so it builds identically on a laptop and in a container, and drops the engine download from the image entirely.

One validation schema, two consumers

The Zod schemas live in a shared package imported by both the React form and the Express middleware, so the client cannot accept what the server rejects. The EMI function is shared for the same reason: the instalment shown live on the form is computed by the same code the engine scores against, so the number on screen is provably the number assessed.

Smaller calls

- **Every failing field returns at once, with one message each.** Chained refinements would report three complaints for a single typo.
- **Duplicate PAN returns 409 with the existing id** rather than overwriting. The client updates that profile — the decision must be scored against the revenue submitted now, not one left from a prior application.
- **The decision endpoint is idempotent**, so a retry after a dropped connection cannot produce a second, different decision.
- **Liveness and readiness are separate**, so a database blip cannot get a healthy container killed and restarted into the same blip.

3. Where this is weakest

Thresholds are reasoned from published Indian MSME lending practice, not calibrated against observed defaults — unavoidable without a portfolio, but it makes every number a hypothesis. Three gaps matter more than the rest:

- **Revenue is unverified.** Self-reported, with no bank-statement analysis or GST cross-check. The largest gap between this and a real system: the most important input is the one nobody checked.
- **Revenue is scored, not profit.** A retailer on 8% net margin and a services firm on 40% are treated identically by the affordability factor. One extra form field would improve the scorecard more than any refinement of the existing thresholds.
- **Existing obligations are invisible.** True FOIR counts all current debt service; this sees only the loan applied for, so it overstates affordability for anyone already carrying debt.

The sector factor is a placeholder, capped at ± 20 — a twentieth of the affordability factor's spread — precisely so it can nudge a borderline file but never decide one on a guess.

4. With more time

Immediately: rate limiting on the decision endpoints (Redis is already in the stack for BullMQ, so this is small — cut only to keep scope honest); idempotency keys on intake, since the decision is idempotent but the application create is not; authentication and per-applicant authorisation, as every endpoint is currently public; and money stored as minor units in `BIGINT` rather than `NUMERIC` converted at the edge.

Next: real signals — a bureau pull, GST filing history, and bank-statement analysis for verified revenue and existing obligations. That is what turns a plausible scorecard into a usable one. Then risk-based pricing, replacing the flat 18% with a rate priced off the risk band so rate and score are solved together rather than one assumed to compute the other. Then a counter-offer engine: `findMaxApprovableAmount()` already binary-searches the engine itself rather than inverting the scorecard by hand, so it cannot drift when thresholds change — turning a decline into "*₹8L over 24 months would be approved*" is mostly UI work from here, and converts rejections into offers.

Structurally: champion/challenger scorecard versioning — decisions already carry `engineVersion`, so the next step is running a candidate in shadow against live traffic and backtesting a retune before it decides a real application. Alongside that, calibrating every threshold against observed default rates rather than first principles; `config/scoring.ts` is structured so that is a data change, not a rewrite. Reason codes and the full scoring trace are already persisted, so generating a regulator-facing adverse-action notice is the remaining piece.